T3000 API & Protocol

T3000 WebView System Architecture · built Nov 13, 2026

T3000 WebView System Architecture

Overview

The T3000 WebView is a modern web-based building automation system that provides real-time monitoring and control of HVAC devices through a hybrid desktop/web architecture.

Architecture Type: Hybrid Desktop Application with Web Frontend **Technology Stack:** Rust Backend + Vue.js/React Frontend + SQLite Database **Communication:** BACnet Protocol + WebSocket + REST API

System Components

Frontend Layer (Client)

The web interface you interact with runs in your browser or embedded WebView.

- **Vue.js Application**: Main user interface for device control
- **React Modules**: Database viewer and developer tools
- **Quasar Framework**: Responsive UI components

Backend Layer (Rust API Server)

The backend server runs on port 9103 and handles all communication.

- **REST API**: Device management, configuration, and database queries
- **WebSocket**: Real-time updates and control messages
- **BACnet Bridge**: Connects to physical devices via FFI layer

Data Layer (SQLite Databases)

Three databases store all system data:

- **T3000.db**: Main configuration, devices, users, settings
- **webview_t3_device.db**: Device data cache (inputs, outputs, variables)
- **T3000_Trendlog.db**: Historical trend data and time-series records

Device Layer (BACnet Network)

Physical building controllers communicate via BACnet/IP protocol:

- T3-BB (BACnet Building Controller)
- T3-TB (Thermostat Controller)
- T3-LB (Lighting Controller)

How Device Loading Works

Step 1: Network Scan

When you open the app, it scans the network for BACnet devices by broadcasting a "Who-Is" message.

Step 2: Device Discovery

Devices respond with "I-Am" messages containing their IP address, device ID, and model information.

Step 3: Data Retrieval

When you select a device:

1. Frontend sends a SELECT_PANEL message via WebSocket
2. Backend connects to the device's IP address
3. BACnet layer reads all data points (inputs, outputs, variables)
4. Data is cached in the database
5. Frontend receives the data and displays it in tables/grids

Step 4: Real-Time Updates

Changes are monitored and updated automatically:

- Device value changes trigger WebSocket broadcasts
- Frontend updates the UI without page refresh
- Database cache stays synchronized

User Interface

Device Tree

Browse all discovered devices in a hierarchical tree view organized by location or type.

Data Points

View and edit device data in organized tabs:

- **Inputs**: Temperature sensors, humidity, pressure
- **Outputs**: Relays, dampers, valves
- **Variables**: Setpoints, calculations, custom values
- **Programs**: Control logic and automation
- **Schedules**: Time-based control
- **Graphics**: Visual building layouts

Trend Logs

Historical data visualization with customizable time ranges and multiple data series.

Configuration

Network Settings

Configure IP addresses, subnets, and protocol ports in Settings → Network.

User Management

Create operator and administrator accounts with role-based permissions.

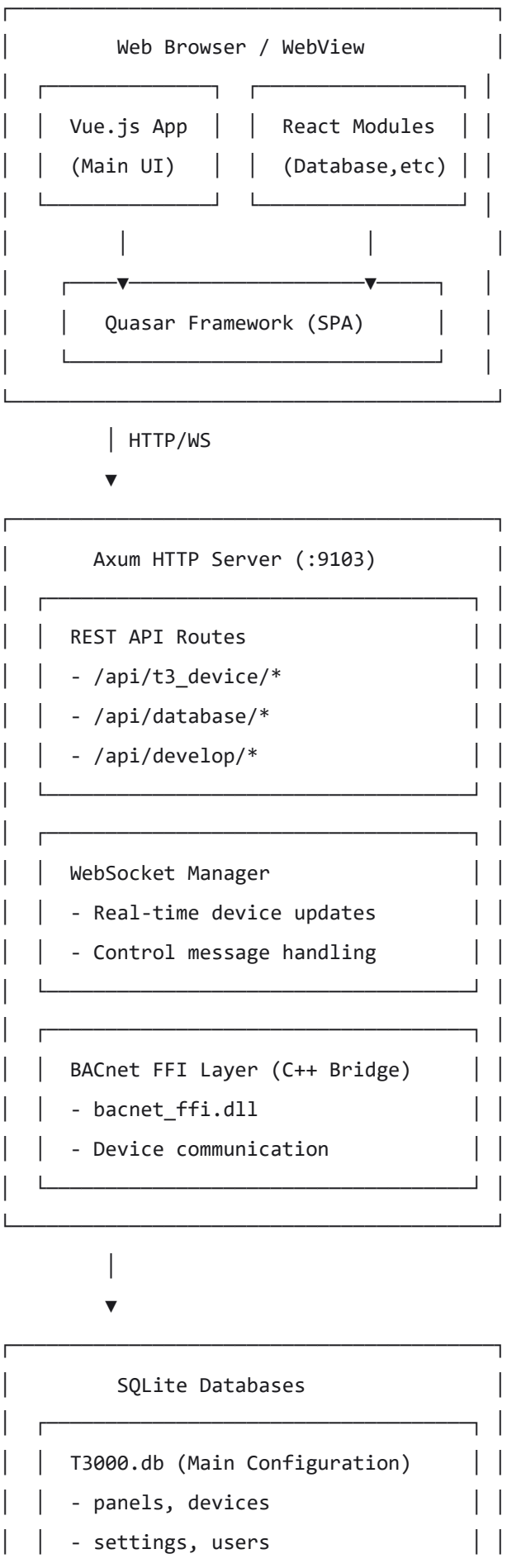
Backups

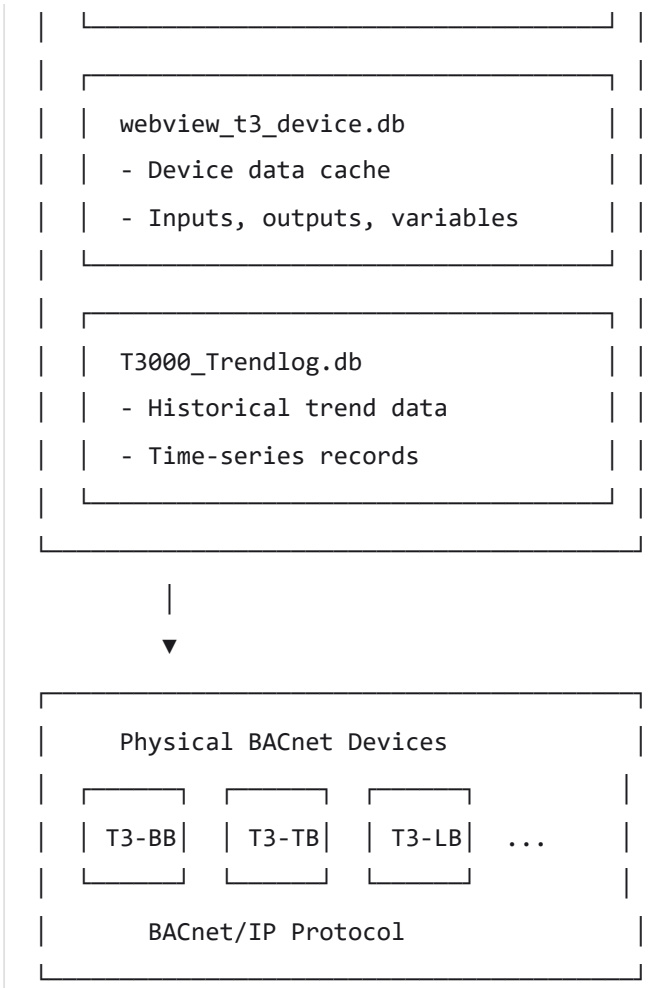
Enable automatic daily backups to protect your configuration and historical data.

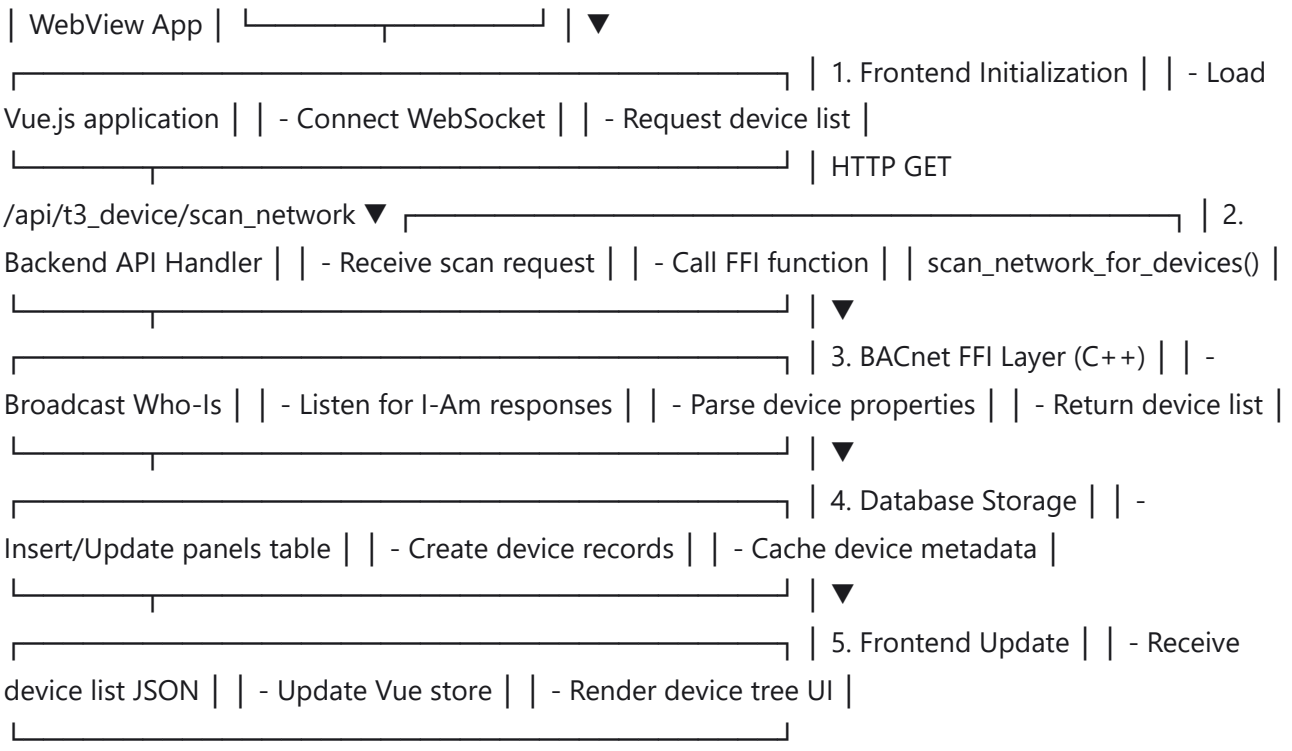
Detailed System Architecture

Component Architecture Diagram

TEXT

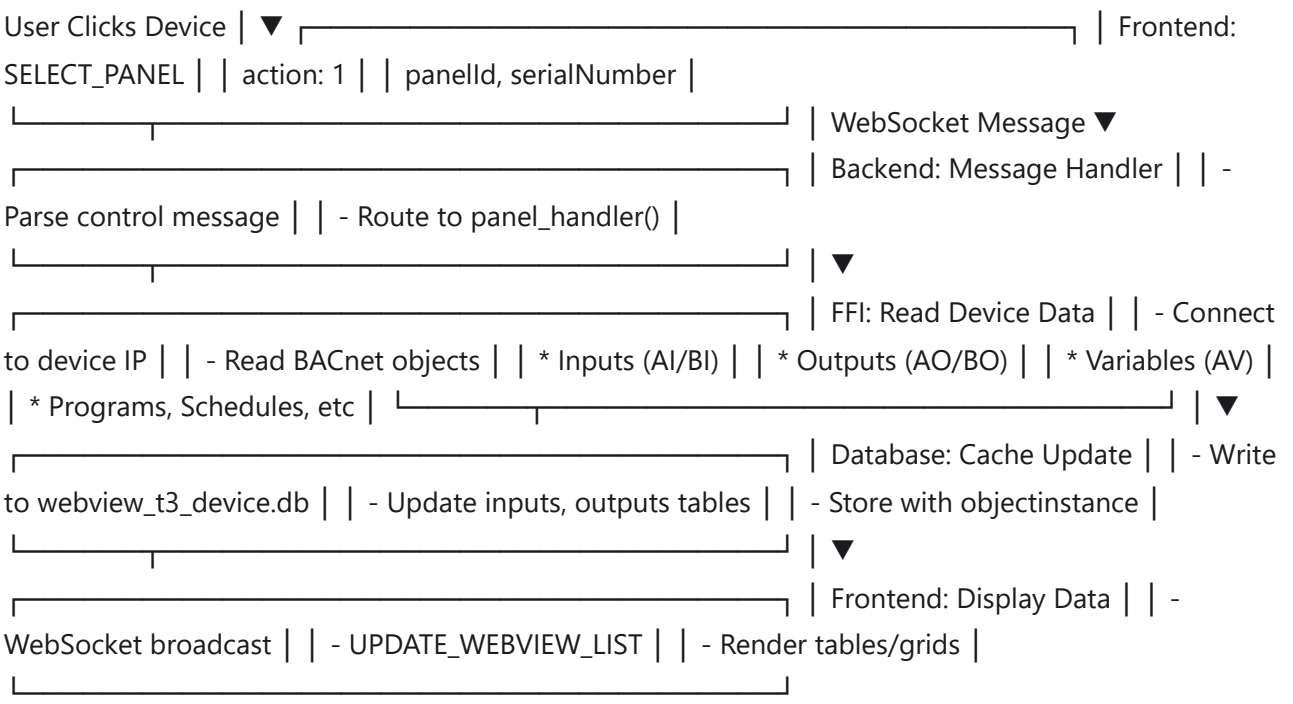

Web Browser / WebView
Vue.js App
(Main UI)
React Modules
(Database,etc)
Quasar Framework (SPA)
HTTP/WS
Axum HTTP Server (:9103)
REST API Routes
- /api/t3_device/*
- /api/database/*
- /api/develop/*
WebSocket Manager
- Real-time device updates
- Control message handling
BACnet FFI Layer (C++ Bridge)
- bacnet_ffi.dll
- Device communication
SQLite Databases
T3000.db (Main Configuration)
- panels, devices
- settings, users


webview_t3_device.db
- Device data cache
- Inputs, outputs, variables
T3000_Trendlog.db
- Historical trend data
- Time-series records
Physical BACnet Devices
T3-BB
T3-TB
T3-LB
...
BACnet/IP Protocol


WebView App
1. Frontend Initialization
- Load Vue.js application
- Connect WebSocket
- Request device list
HTTP GET /api/t3_device/scan_network
2. Backend API Handler
- Receive scan request
- Call FFI function
scan_network_for_devices()
3. BACnet FFI Layer (C++)
- Broadcast Who-Is
- Listen for I-Am responses
- Parse device properties
- Return device list
4. Database Storage
- Insert/Update panels table
- Create device records
- Cache device metadata
5. Frontend Update
- Receive device list JSON
- Update Vue store
- Render device tree UI

TEXT

```
### Device Data Loading Flow
```



TEXT

```
---

## Communication Protocols

### 1. Frontend ⇔ Backend

**REST API (HTTP)**
- Device scanning
- Configuration management
- Database queries
- File operations

**WebSocket (Real-time)**
- Control messages (action-based)
- Live data updates
- Device status changes
- Trend data streaming

### 2. Backend ⇔ Devices

**BACnet/IP Protocol**
- Who-Is / I-Am (device discovery)
- Read-Property (data retrieval)
- Write-Property (control commands)
- Subscribe-COV (change notifications)

---

## Control Message System

### Message Structure
```json
{
  "action": <number>,
  "source": 0|1,          // 0=T3000, 1=WebUI
  "panelId": <number>,
  "serialNumber": <number>,
  "entryType": <number>,
```
```

```
  "data": { ... }
}
```

Common Actions

Action	Name	Purpose
1	SELECT_PANEL	Load device data
2	SAVE_ENTRY	Write single value
16	UPDATE_WEBVIEW_LIST	Bulk write
17	READ_SINGLE_ENTRY	Read one entry

Database Schema

T3000.db

SQL

```sql
-- Device registration
panels (
  id, serial_number, product_name,
  ip_address, port, location
)

-- User management
users (
  id, username, password_hash, role
)

-- System settings
application_settings (
  key, value, category
)
```

webview_t3_device.db

```SQL
-- Device data cache
inputs (
  id, panel_id, number, label,
  value, units, auto_manual
)

outputs (
  id, panel_id, number, label,
  value, auto_manual, range
)

variables (
  id, panel_id, number, label,
  value, units, auto_manual
)
```

Deployment Architecture

Development Mode

```TEXT
Frontend (Quasar Dev Server)
  ↓ Port 3004
  ↓ Proxy to backend
Backend (Rust cargo run)
  ↓ Port 9103
  ↓ SQLite files in ./Database
Devices (Local network)
```

Production Mode

```
TEXT
```

```
Electron/Tauri Desktop App
  ├── Bundled Frontend (SPA)
  ├── Embedded Rust Backend
  └── SQLite in %AppData%/T3000
       ↓
  BACnet Network
```

Key Design Patterns

1. Event-Driven Architecture

- WebSocket for real-time updates
- Publisher/subscriber pattern
- Async/await for non-blocking I/O

2. Caching Strategy

- Database cache for offline access
- In-memory state for active devices
- Background sync with physical devices

3. State Management

- Frontend: Vue Pinia stores
- Backend: Rust Arc<Mutex> for shared state
- Database: SQLite transactions

4. Error Handling

- Frontend: Try-catch with user notifications
- Backend: Result<T, E> with proper propagation
- FFI: Safe error conversion across boundaries

Performance Considerations

Optimization Points

1. **Device Scanning**: Parallel network scans
2. **Data Loading**: Batch reads via bulk messages
3. **UI Updates**: Debounced re-renders
4. **Database**: Indexed queries on objectinstance
5. **WebSocket**: Message compression for large datasets

Scalability Limits

- Max devices per network: ~250 (BACnet limitation)
- Max concurrent connections: ~100 WebSocket clients
- Database size: Tested up to 10GB trendlog data

Security Model

Authentication

- User login with bcrypt password hashing
- Session tokens for API access
- Role-based permissions (admin/user)

Network Security

- BACnet authentication (optional)
- HTTPS for production deployment
- CORS configuration for web access

Future Enhancements

Planned Features

1. **Cloud Sync**: Multi-site management
2. **Mobile App**: React Native companion
3. **AI Analytics**: Predictive maintenance
4. **RESTful BACnet**: Bridge to cloud services

Technical Debt

1. Migrate from mixed Vue/React to single framework
2. Implement GraphQL for flexible data queries
3. Add comprehensive integration tests
4. Improve FFI error handling with detailed codes

Getting Started for Developers

Prerequisites

```shell
## Rust toolchain
rustup install stable

## Node.js & npm
nvm install 18

## Quasar CLI
npm install -g @quasar/cli
```

Development Setup

```shell
## 1. Start backend
cd api
cargo run

## 2. Start frontend
npm run client-dev

## 3. Access UI
open http://localhost:3004
```

Project Structure

```
TEXT
```

```
T3000Webview9/
├── api/              # Rust backend
│   ├── src/
│   │   ├── t3_device/     # Device management
│   │   ├── t3_develop/    # Developer tools
│   │   └── database_management/
│   └── build/        # FFI build outputs
├── src/              # Frontend source
│   ├── t3-vue/       # Vue components
│   └── t3-react/     # React modules
├── docs/             # Documentation
└── Database/         # SQLite files
```

Troubleshooting Guide

Common Issues

Device not detected

- Check BACnet router configuration
- Verify IP subnet matches
- Ensure Who-Is broadcasts are allowed

WebSocket disconnects

- Increase timeout in backend config
- Check firewall settings on port 9103
- Verify network stability

Database locked errors

- Only one writer at a time (SQLite limitation)
- Use WAL mode for concurrent reads
- Close connections properly in code

References

- BACnet Protocol Standard

- Axum Web Framework
- Quasar Framework
- SQLite Documentation